PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Contents

1. Project Overview 3
2. Background & Business Objective 3
3. Product Purpose 4
4. System Functionalities 5
5. User Stories & Use Cases 7
6. Features Included (Scope Definition) 9
7. Features Not Included (Scope Control) 9
8. Assumptions & Constraints 9
9. Risks & Questions Throughout Development 10

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: SmartFlow Orders

Version: 1.0

1. Project Overview

- **Problem Statement:** Online stores offer a vast variety of products, but building a complete, multi-item solution (e.g., a tech setup, a DIY kit, or a themed bundle) requires customers to manually search across different categories, check individual specifications, and mentally verify compatibility. When an item is out of stock or exceeds the budget, the entire manual process breaks down, leading to frustration, abandoned carts, and repetitive customer service queries.
- **Target Domain:** General E-commerce platforms, focusing on multi-item product bundling, intelligent cross-category discovery, and automated customer service workflows.
- **Proposed Solution Summary:** SmartFlow Orders eliminates the hassle of complex online shopping by converting natural language prompts into fully customized carts. It handles the logistics of stock availability and budget constraints behind the scenes, ensuring the user receives a cohesive bundle without ever having to troubleshoot individual items or links.

2. Background & Business Objective

- **Background of the Problem:** For decades, e-commerce has relied on a search-and-retrieve model where the burden of "assembly" lies entirely on the shopper. To build a complex solution—like a home recording studio or a beginner's gardening kit—a user must manually navigate disparate categories, cross-reference technical specifications, and keep a mental tally of their budget. This linear process is highly inefficient, requiring significant time and specialized product knowledge. If a single component is unavailable, the user must restart the entire logic chain, often resulting in "decision fatigue" and abandoned shopping sessions.

© UMHackathon 2026
Email: umhackathon@um.edu.my

- **Importance of Solving This Issue:** By collapsing a multi-hour research and assembly process into a few seconds of natural language processing, we reclaim the user's most valuable resource: *time*. Solving this issue removes the "knowledge barrier" to shopping; a customer no longer needs to be a technical expert to build a compatible tech setup or a professional kit. For businesses, this means higher conversion rates, reduced reliance on human support staff for inventory queries, and a transition from selling "individual parts" to selling "complete outcomes."
- **Strategic Fit / Impact:** SmartFlow Orders align perfectly with the Agentic AI movement by transforming LLMs from passive chat interfaces into active workflow orchestrators. By leveraging high-reasoning models (like Gemini) and integrating with backend inventory systems, our project demonstrates a real-world application of Function Calling and Stateful Decision-Making. It fits into the broader ecosystem of "Intent-to-Action" commerce, where AI doesn't just suggest products but autonomously builds and optimizes complex transactions, turning a standard database into an intelligent, responsive agent.

3. Product Purpose

3.1. Main Goal of the System: To provide an intelligent, agentic procurement assistant that autonomously transforms high-level user intent into complete, verified, and budget-optimized product ecosystems ready for checkout.

3.2. Intended Users (Target Audience):

- "Goal-Oriented" Consumers: Shoppers who have a specific end-result in mind (e.g., "Set up a home gym") but lack the technical knowledge to select compatible components.
- Budget-Conscious Buyers: Users who need to maximize the value of a fixed total amount and require an AI to "puzzle-fit" the best items into their cart.
- DIY Project Starters: Hobbyists embarking on multi-step builds (PC building, gardening, home repair) where missing a single small part or "spec" ruins the whole project.

© UMHackathon 2026

- Small Business Buyers: Owners who need to kit out an office or shop quickly without spending hours comparing individual product listings.
- E-commerce Support Agents: Internal staff who use the tool to instantly provide "Smart Alternatives" to customers when a specific item is unavailable.

4. System Functionalities

4.1. Description: SmartFlow Orders operates as an ***agentic fulfillment engine*** that translates natural language intent into complex, multi-item product solutions. By leveraging an ***Intent-to-Cart*** architecture, the system captures functional relationships between products, verifies technical compatibility, and optimizes for inventory availability and budget constraints in real-time.

4.2. Key Functionalities:

- **Intent-Driven Bundling:** Translates broad human-language goals (e.g., "I need a professional podcasting setup for two people") into a curated, high-fidelity list of compatible products.
- **Dynamic Inventory Swapping:** Automatically identifies out-of-stock items or budget overages and intelligently substitutes them with "best-fit" alternatives without breaking the bundle's integrity.
- **Stateful Conversational Refinement:** Facilitates continuous modification of the cart through natural dialogue (e.g., "Make the keyboard wireless" or "Keep the total under $500"), maintaining the state of all other items.
- **Structured Checkout Export:** Converts the finalized AI-curated selection into a structured JSON payload ready for direct injection into e-commerce checkout APIs or database fulfillment rows.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: We utilized ***Z AI (ILMU-GLM-5.1)*** as the core brain of SmartFlow Orders. This model was selected for its high ***Reasoning-to-Action ratio*** and its robust ***Function Calling*** capabilities. Unlike standard LLMs that only generate text, Z AI excels at

© UMHackathon 2026

understanding when it needs to call out inventory and budget tools to make real-world decisions. Its ability to maintain long-context coherence ensures that when it swaps an item, it remembers the original user intent and the constraints of the entire "ecosystem."

4.3.2. Prompting Strategy: Our team employed a ***Multi-Step Agentic Prompting*** strategy, utilizing ***Chain-of-Thought (CoT)*** to ensure logical consistency.

- **The Reason:** Instead of asking the model to "Build a Cart" in one go, we prompt it to ***Plan, Validate, and Assemble***. First, the model plans the components needed; second, it calls tools to validate stock; third, it assembles the final bundle. This strategy prevents "logic drift" and ensures that every item in the final output is technically compatible and financially viable.

4.3.3. Context & Input Handling: SmartFlow Orders is designed to handle messy, "unstructured" human requests (e.g., a long paragraph describing a dream gaming room).

- **Handling Oversized Inputs:** We accept inputs up to ***128k tokens*** (leveraging Z AI's deep context window). However, to maintain high speed, we use a ***Context-Filtering Agent*** to strip away conversational "noise" and extract a structured ***JSON-based Requirement Map*** (Budget: $X, Style: Y, Essential Items: Z). If an input exceeds our processing threshold, the system prioritizes the "hard constraints" (Budget and Compatibility) while truncating descriptive fluff to avoid latency.

4.3.4. Fallback & Failure Behavior: To ensure professional user experience, we have implemented a ***Graceful Degradation*** mechanism:

- **Retry on Hallucination:** If Z AI returns an item ID that doesn't exist in our mock database, a "Self-Correction" loop triggers a second pass with a narrowed search scope.

© UMHackathon 2026

- **Partial Solution Path:** If a user's budget is impossible to meet, the system doesn't fail; it returns a "Best Effort" cart, highlights which items are over-budget, and explains *why* the swap was made.

- **Human-in-the-Loop Escalation:** For internal e-commerce agents, if the AI cannot find a suitable swap after 3 attempts, it flags the ticket for a human manager, providing a summary of what it tried so the human doesn't have to start from scratch.

5. User Stories & Use Cases

- **User Stories:**
  - **The "Goal-Oriented" Shopper:** "As a hobbyist, I want to describe a project goal in plain English so that I can receive a complete list of compatible parts without having to research technical specifications myself."

  - **The Budget-Conscious Buyer:** "As a small business owner, I want to provide a strict budget for an office setup so that AI can automatically find the best-performing products that fit within my price limit."

  - **The Support Agent:** "As a customer service representative, I want to instantly find compatible replacements for out-of-stock items so that I can resolve customer complaints in one interaction without manual database searching."

- **Use Cases (Main Interactions):**

  - **Intent-to-Bundle Flow:** The user enters a descriptive prompt (e.g., "Set up a beginner vlogging kit"). SmartFlow Orders parses the intent, identifies necessary categories (Camera, Mic, Tripod), calls the inventory tool to find compatible items, and renders a complete, unified cart.

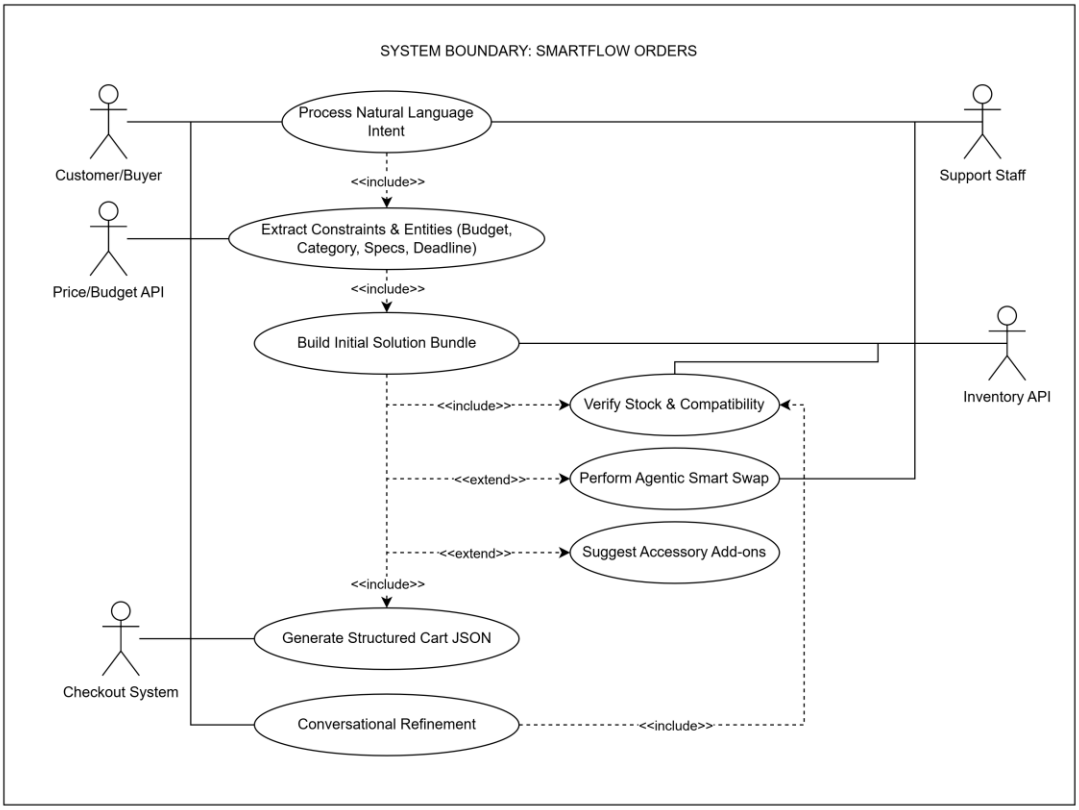
  - **Intelligent Swap Flow:** If a user modifies a cart (e.g., "Upgrade the camera to 4K") and the new total exceeds the budget, the system triggers the "Smart Swap" logic.

© UMHackathon 2026

It identifies non-essential items in the cart (like a premium tripod) and replaces them with budget-friendly alternatives to maintain the total price constraint.

- **Compatibility Check Flow:** When a user adds an item manually, the AI agent performs a background "Logic Check" against existing items (e.g., "Checking if this lens fits that camera body"). If a mismatch is found, it alerts the user and suggests the correct adapter or alternative.

- **Use Case Diagram:**



© UMHackathon 2026
Email: umhackathon@um.edu.my

6. Features Included (Scope Definition)

- **Natural Language Bundle Architect:** A conversational interface that interprets complex, multi-item intent and converts it into a cohesive product list.
- **Agentic Logic Engine (Smart Swap):** Real-time monitoring of inventory and budget; the system autonomously identifies conflicts and suggests technically compatible alternatives.
- **Stateful Dialogue Memory:** The ability for the AI to retain the context of the current "cart" during refinement (e.g., changing one item without losing the rest of the configuration).
- **Multi-Constraint Optimization:** Simultaneous balancing of three variables: User Intent (Style/Goal), Technical Compatibility (Specs), and Financial Limits (Budget).
- **Structured JSON Export:** A "ready-to-checkout" data output that maps the AI's decisions into a format usable by external databases or e-commerce APIs.

7. Features Not Included (Scope Control)

- **Live Payment Processing:** The system generates a validated cart but does not handle actual credit card transactions or financial clearing.
- **Real-World Inventory Sync:** For the hackathon version, the system interacts with ***a mock database (JSON)*** rather than a live, production-level warehouse API (e.g., real Shopify/Amazon backends).
- **Logistics & Last-Mile Tracking:** The scope ends at the "Checkout-Ready" stage; it does not include shipping calculations, tracking numbers, or delivery status.
- **Authentication & Profiles:** This version is a functional prototype focused on the agentic workflow; it does not support saved user accounts, password recovery, or order history [Initial Version].

8. Assumptions & Constraints

- **LLM Cost Constraint:** We estimate an average user session (intent extraction + 3 tool calls + final bundle generation) will cost approximately ***3,000–5,000 tokens*** using Z AI (GLM-4).
    - **Optimization Decision:** To keep inference costs manageable at scale, we implemented ***System-Level Constraint Extraction***. Instead of sending the entire

raw chat history back to the LLM for every refinement, we maintain a small "State JSON" (containing only the current budget and active item IDs) to use as context, significantly reducing token consumption per turn.

- **Technical Constraints:** For the prototype, the product database is a structured JSON mock-up. Live synchronization with dynamic warehouse inventory systems (like SAP or Oracle) is not supported in this version. Data output is limited to ***Standardized JSON payloads*** for checkout systems.
- **Performance Constraints:** To maintain high-speed reasoning for "Smart Swapping," we limit the number of simultaneous alternative searches to ***5 per item***. This ensures that the user receives a response within seconds rather than waiting for an exhaustive search of the entire catalog.
- **User Input:** The system is an ***"Agent-in-the-Loop"*** solution; it requires a clear initial human prompt to define the goal and requires final human approval of the cart before the JSON payload is exported to the checkout API.

9. Risks & Questions Throughout Development

- **Compatibility Accuracy:** How do we ensure the AI correctly interprets technical specifications (e.g., ensuring a specific camera lens *actually* fits a specific mount)?
    - *Mitigation:* We use ***RAG (Retrieval-Augmented Generation)*** logic to feed the AI specific "Compatibility Tables" from our database rather than relying on the model's internal general knowledge.
- **Budget "Dead-Ends":** What happens if the user's budget is mathematically impossible for the requested items?
    - *Question:* Should the AI refuse to build the cart, or build the "cheapest possible" version and flag the overage? (Currently, we choose the "Best Effort + Flag" approach).
- **Hallucinated Products:** How do we prevent the AI from "inventing" a product that doesn't exist in the store?

© UMHackathon 2026
Email: umhackathon@um.edu.my

- *Mitigation:* We use ***Strict Tool Output Validation***. The final cart is cross-referenced against the database IDs one last time before the UI renders, ensuring 100% data integrity.

- **Prompt Ambiguity:** What happens if the user says "I want a set for a good time"?
  - *Strategy:* If the intent is too vague, the AI is programmed to pause the workflow and ask for the ***Primary Category*** or ***Budget*** before proceeding with any tool calls.

© UMHackathon 2026
Email: umhackathon@um.edu.my